ENGINEERING STATUS DOC

Mortgage QC Engine — Implementation Audit

Reconcile · QC · Disposition

One view of the three evaluation layers built so far — how they connect and overlap, how a loan's verdict moves through them, and the case for pausing here before the next build phase.

Companion to output/THESIS.md, output/ROADMAP.md, .specify/memory/constitution.md

1. The engine in one picture

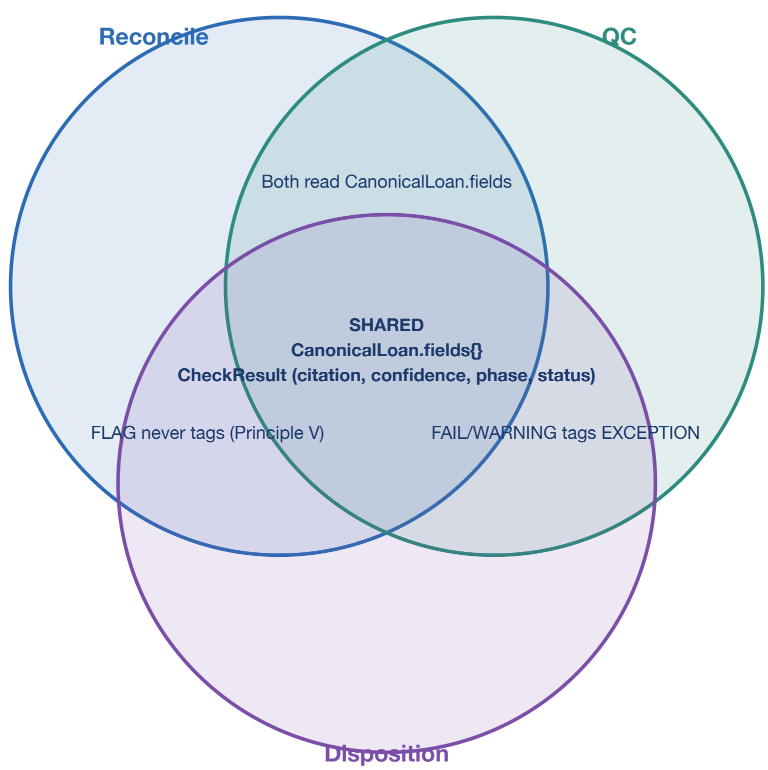
Reconcile compares the closing document against the system of record and flags where they disagree. **QC** applies policy and math rules to the document itself, regardless of what the system says. **Disposition** composes both into the one verdict a human actually sees. They run in a fixed sequence, but every layer reads from the same field catalog and writes the same audit shape.

Reconcile	QC	Disposition
Source Envelope (001b)	Field Catalog (001a)	review_reason tagging (004)
agree_categorical	predicate: is_true / is_present	review_reasons set (union)
agree_numeric	ratio_threshold: ltv / dti / field_value	AUTO_CLEARED or NEEDS_REVIEW
FLAG verdict (info only)	PASS or FAIL verdict	

The engine's three layers — what each one owns, from the loan's fields to one composed verdict.

2. How the layers overlap

They are not three independent programs. They share one loan model and one audit shape, and they meet at named seams where a status computed by one layer becomes an input the next layer reads — never recomputes.



The overlap — one loan model and one audit record at the centre; the seams are where a status becomes another layer's input.

3. The layers side by side

Layer	Purpose	Check kinds	Verdicts possible	Real-world scale	Spec(s)
Reconcile	Doc (truth) vs. system-of-record agreement	`agree_categorical`, `agree_numeric`	PASS, FLAG, NEEDS_REVIEW, NOT_APPLICABLE	~402 conditions (INACCURATE 263 + MISMATCH 139)	003c
QC	Policy, presence, and boundary math against the document	`predicate`, `ratio_threshold`	PASS, FAIL, NOT_APPLICABLE	~2,937 + ~853 conditions (MISSING/POLICY/UNSIGNED/EXPIRED/INCOMPLETE + LTV/DTI/thresholds)	003a, 003b
Disposition	One composed per-loan verdict	*(composes the other two — no kind of its own)*	AUTO_CLEARED, NEEDS_REVIEW	1 verdict per loan, from an open `review_reasons` tag set	004

4. How the layers connect

Reconcile and QC each run independently against the same loan; they meet only at Disposition, which reads what already happened and never re-derives it.

From	To	What flows / why
Reconcile	Disposition	A `FLAG` is excluded from `review_reasons` by construction — it never blocks auto-clear (Principle V)
QC	Disposition	A QC-phase `FAIL/WARNING` is tagged `EXCEPTION`
Confidence gate (cross-cutting)	Disposition	A `PASS` resting on `doc_confidence` below the 0.80 floor is downgraded to `NEEDS_REVIEW`, tagged `LOW_CONFIDENCE`
All three	Audit export	Every `CheckResult` — citation, verdict, reason — surfaces to the Excel review sheet Kayla reads

5. How a loan's verdict moves

The loan's own record grows as it passes through the pipeline — nothing is thrown away, and every stage adds fields rather than replacing them.

CanonicalLoan
loan_id, fields{doc, sources}, facts{}

CheckResult (per check)
+ phase, status, citation, doc_confidence

RunResult (per loan)
+ flags, qc_failures, needs_review, review_reasons

Disposition
+ AUTO_CLEARED or NEEDS_REVIEW (final)

Excel export
collects from all loans into Summary + Check Detail sheets

One loan's evaluation, stage by stage — the record grows; nothing already computed is recomputed downstream.

6. The shared foundation

All three layers stand on the same guarantees — which is what makes "same loan, same verdict, every time" provable rather than asserted.

Shared element	What it provides across all three layers
Pure function, no runtime LLM	No network, no model, no wall-clock — same inputs always produce the same `CheckResult`
Decimal money math, `ROUND_HALF_EVEN`	No float ever touches a pass/fail boundary (the exact class where a weaker model bought back a 98%-LTV loan in the G3 bake-off)
Field catalog governance (001a, 377 entries)	A check referencing an unknown field fails loudly at build time, not silently at runtime
Citation-traceable `CheckResult`	Every doc-sourced value traces to a document name, page, and highlighted segment
Zero-regression digest discipline	Every one of the 8 specs proves its change is additive before it merges — the digest has held or been deliberately, provably extended since 001b

7. The case for holding here before the next build phase

Each layer is independently proven: 128 tests, a 1,000-run bit-exact digest, and 25 of 25 known planted defects reproduced exactly. But 004 is the first feature that composes all three layers into one per-loan verdict — which makes it the natural checkpoint to hold at, and build the next phase's eval gate against, before extending the check-kind vocabulary further.

Why pause here before 005 and beyond:

- **One eval gate would govern every future check-kind.** 005 turns the existing `eval_synth` scorer into a CI promotion gate, so nothing shipped after this point can bypass it.
- **006 and 008 both attach directly to 004's seam.** The confidence gate and the exception queue build on `disposition`, which is already proven equivalent to the pre-existing `auto_cleared` boolean by construction — safer to build on top of a composition that's proven, not assumed.
- **The gap list is short and named, not hidden.** 5 of 25 known defects are genuine doc-vs-doc comparisons with no check-kind built yet; the applicability-gating rules (which checks apply to which program) are hand-derived, not yet SME-validated.
- **Handoff-ready.** Anyone inheriting this repo next — Monish's team, for the industrial build-out — gets a fully tested, fully proven core, not a partially-verified one.
- **The trade-off.** Every proof above holds against 5 synthetic loans with 20 of 25 known defects wired — not yet real loan volume, and the gating rules are derived from this one small dataset, not signed off by an SME. Read "proven correct" here as "proven correct on the fixtures that exist today," not yet "proven correct at production scale."

8. The unifying principle

The same idea repeats at every layer, which is why they compose without needing per-layer glue code: **tag generically by what already happened; never rebuild logic per case.**

- **Reconcile owns the truth-vs-system comparison.** A `FLAG` is informational by construction — it is never fed back into QC as a fact.
- **QC owns policy and math.** Pass/fail is decided once, against the document alone, never against what the system says.
- **Disposition owns composition only.** It reads `phase` and `status` that already exist; a future fifth check-kind is tagged automatically the moment it produces a QC-phase `FAIL`, with zero changes to the composition code.
- **A person approves what needs review.** Determinism computes the verdict; disposition tags *why* it needs a human; the reviewer still decides.

> **The one-line takeaway**
>
> Three deterministic layers, one CheckResult shape, one digest — a future check-kind is tagged automatically the moment it exists, with zero changes to the layer above it.

9. Summary

- **8 specs complete, in sequence (000 → 004).** 128 tests passing, zero unintended regressions since 001b — every legitimate change is proven additive before it merges.
- **Two independent proofs, not one.** `harness.py`'s 1,000-run bit-exact digest (`a3f702c1...`) proves the engine is deterministic; `verify_against_defects.py`'s 25/25 match proves the fixture data is right — different claims, both green.
- **Hold at 004, build 005 next.** The gaps that remain — 5 doc-vs-doc defects, hand-derived gating rules — are explicit and tracked, not silently absorbed into "done."